

AND-Popcount FP8 MLB-MAC: Eliminating the Bipolar Correction Overhead for Energy-Efficient DNN Inference

Shreyas S. Kulkarni
IIIT Bangalore
ShreyasS.Kulkarni@iiitb.ac.in

Grover Heer
IIIT Bangalore
Heer@iiitb.ac.in

Madhav Rao
IIIT Bangalore
mr@iiitb.ac.in

Abstract—This paper presents an FP8 Multi-Level Binary MAC (MLB-MAC) accelerator for energy-efficient DNN inference that operates natively in the FP8 (E4M3) format. Prior MLB-MAC work [1] computes dot products with an XNOR-popcount core that maps operands into a bipolar $\{-1, +1\}$ space. Applying XNOR to data that is not natively bipolar introduces a systematic offset $(2x_i - 1)(2x_j - 1) = 4x_i x_j - 2x_i - 2x_j + 1$, which requires dedicated C_1 , C_2 , and C_3 correction accumulators, shift-add constant multipliers ($30 \times C_1$, $30 \times C_2$, $225 \times C_3$), and a 23-bit wide-adder correction block, eroding the area and power advantage the bitwise core was intended to provide. We observe that FP8 (E4M3) mantissas—including the recovered hidden bit—are strictly unsigned binary fractions where a logical 0 corresponds exactly to a mathematical 0. Replacing the XNOR gate with an AND gate removes the mismatch at its source: $\text{AND}(x_i, x_j) = x_i \cdot x_j$ for $x_i, x_j \in \{0, 1\}$, with no offset. This drives C_1 , C_2 , and C_3 to zero, allowing the entire correction block—including its shift-add multipliers and accumulator trees—to be deleted outright. The AND-popcount core further replaces the per-bit-plane `basis_multiplier` module (a 16-entry LUT with a barrel-shift pipeline and fallback 8×9 -bit multiplier) with a zero-gate multiplier that left-shifts the popcount result by a static $(i + j)$ bit-positions. We combine this core with a parameterizable Block Floating Point (BFP) front end, an input time-multiplexing FSM for resource sharing, and a sign-magnitude output combiner. We synthesize the design using Yosys with the NanGate 45nm Open Cell Library at a target clock period of 10ns, and verify RTL correctness with 61 self-checking test vectors per N value. A Quantization-Aware Training (QAT) sweep on ResNet-20/CIFAR-10 across $N \in \{9, 25, 32, 64, 128, 256, 512\}$ with exponent-sorted blocking characterises the accuracy–area trade-off.

Index Terms—DNN accelerators, FP8, E4M3, MAC array, AND-popcount, multi-level binary, block floating point, quantization-aware training, energy efficiency, RTL verification

I. INTRODUCTION

Low-precision arithmetic has become central to energy-efficient deep neural network (DNN) inference. The 8-bit floating-point format FP8, in particular the E4M3 variant [2], [3], is increasingly adopted as a balance between dynamic range and hardware cost. Within a DNN accelerator, the dominant source of energy and area is the multiply-accumulate (MAC) array that computes the dot products underpinning convolution and fully-connected layers.

Multi-Level Binary (MLB) representation decomposes a quantized value into a linear combination of binary bit-planes,

and MLB-MAC architectures exploit this decomposition to compute dot products with bitwise XNOR-popcount logic instead of full multiplier trees [1]. Because XNOR is a bipolar operator defined over $\{-1, +1\}$, the operands must first be remapped into that space. Applying XNOR to data that is not natively bipolar introduces a systematic mathematical offset. Concretely, mapping an unsigned bit $x \in \{0, 1\}$ to bipolar form $b = 2x - 1 \in \{-1, +1\}$, the product of two mapped bits expands as:

$$b_i \cdot b_j = (2x_i - 1)(2x_j - 1) = 4x_i x_j - 2x_i - 2x_j + 1. \quad (1)$$

Summing over N lanes and B^2 bit-plane pairs, the $-2x_i$, $-2x_j$, and $+1$ terms accumulate into three correction structures C_1 , C_2 , C_3 that existing MLB-MAC designs correct with dedicated accumulator trees, constant multipliers, and wide reduction adders [1]. In the $N=64$ XNOR implementation we developed as a baseline, this correction block requires:

- Three correction accumulators: C_1 (11-bit signed, range $[-960, +960]$), C_2 (11-bit signed), and C_3 (8-bit signed, range $[-64, +64]$).
- Two shift-add constant multipliers computing $30 \times C_1 = (C_1 \ll 5) - (C_1 \ll 1)$ and $30 \times C_2 = (C_2 \ll 5) - (C_2 \ll 1)$ (each 20-bit signed).
- One shift-add constant multiplier computing $225 \times C_3 = (C_3 \ll 8) - (C_3 \ll 5) + C_3$ (17-bit signed).
- A final 23-bit signed adder computing $\text{sum_corr} = \text{MLB4} + 30C_1 + 30C_2 - 225C_3$, followed by an arithmetic right-shift by 2.

This correction hardware partially offsets the area and power savings that the bitwise core was intended to provide.

In this work we revisit this design for FP8 (E4M3) operands. FP8 mantissas—including the recovered hidden “1” bit—are strictly *unsigned* binary fractions: a logical 0 in any mantissa bit position corresponds exactly to a mathematical 0, with sign handled separately by the dedicated sign bit. This native alignment with unsigned semantics means that replacing the XNOR gate with a plain AND gate removes the mismatch entirely:

$$\text{AND}(x_i, x_j) = x_i \cdot x_j, \quad x_i, x_j \in \{0, 1\}, \quad (2)$$

with no offset. Setting $b_i = x_i$ (no bipolar mapping), the correction terms from (1) collapse: $-2x_i$ and $-2x_j$ become

zero (because there is no $2x-1$ mapping), and the $+1$ constant vanishes. Consequently, $C_1 = C_2 = C_3 = 0$, and the entire correction block—accumulators, shift-add multipliers, and 23-bit adder—is deleted outright.

Beyond the correction block, the AND-popcount design eliminates a second source of complexity. In the XNOR design, each of the 16 MLB units contains a `basis_multiplier` module: a two-stage pipelined combinational LUT that maps the pair $(\alpha_{x,i}, \alpha_{w,j})$ to a shift amount $\log_2(\alpha_{x,i} \cdot \alpha_{w,j})$, then barrel-shifts the signed 8-bit XNOR-popcount result by that amount (with a fallback 8×9 -bit multiplier for non-power-of-two α values). Since FP8 bit-planes have fixed power-of-two weights $\{2^0, 2^1, 2^2, 2^3\}$, the AND-popcount design replaces this entire module with a *zero-gate multiplier*: the popcount is simply left-shifted by a *static* $(i+j)$ bit-positions, where i and j index the activation and weight bit-planes. This eliminates 16 instances of the LUT, barrel-shift, and pipeline registers.

To feed the AND-popcount core, we combine it with a lightweight Block Floating Point (BFP) front end that groups N FP8 operands under a single shared exponent. We also implement an input time-multiplexing FSM that routes positive-sign and negative-sign lanes through a single shared MLB 4 array instance over two cycles, halving the array instantiation count. Final floating-point rescaling is deliberately deferred to software, saving additional silicon area. We use a bit-accurate PyTorch emulator and a QAT sweep on ResNet-20/CIFAR-10 to characterise the “outlier cliff” accuracy penalty as a function of block size N , identifying the Pareto-optimal sweet spot between hardware area and inference accuracy.

Contributions

- We identify and eliminate the bipolar-to-unsigned correction overhead in MLB-MAC arrays for FP8 (E4M3) operands by replacing XNOR-popcount with AND-popcount, removing the C_1, C_2, C_3 accumulator trees, the shift-add constant multipliers ($30 \times C_{1,2}, 225 \times C_3$), the 23-bit correction adder, and the 16 per-unit `basis_multiplier` pipeline modules required by the bipolar MLB-MAC design.
- We design a complete, parameterizable FP8-native AND-popcount MLB-MAC RTL pipeline—consisting of a BFP aligner with integrated corner turn, an AND-popcount reduction tree, a zero-gate-multiplier MLB unit, a 4×4 MLB 4 array with three-level combinational reduction tree, a sign-mask FSM for time-multiplexed resource sharing, and a pos/neg output combiner—and generate synthesizable Verilog-2001 across $N \in \{9, 25, 32, 64, 128, 256, 512\}$ using a parameterized Python code generator (`generate_all_N.py`).
- We synthesize both the AND-popcount MLB-MAC and an integer MAC baseline (with the same BFP front end) using Yosys with the NanGate 45 nm Open Cell Library at a 10 ns clock period, reporting power, area, and energy efficiency (IPS/W) via OpenSTA.

- We develop a bit-accurate PyTorch emulator of the hardware’s BFP truncation behavior as a custom `torch.autograd.Function` with a Straight-Through Estimator (STE) [4], implement exponent-sorted blocking as a compiler-level optimization to delay the outlier cliff, run a QAT sweep on ResNet-20/CIFAR-10 from an FP8 baseline, and verify RTL correctness with 61 self-checking test vectors (11 directed corner cases + 50 pseudorandom) per N value.

II. BACKGROUND AND RELATED WORK

A. Multi-Level Binary Representation and MLB-MAC Arrays

Standard integer (uniform) quantization represents a value as $v = s \sum_{k=0}^{B-1} 2^k b_k$, where $b_k \in \{0, 1\}$ and s is a per-tensor scale. Multi-Level Binary (MLB) representation generalizes this by allowing each bit-plane to carry an arbitrary real-valued scaling factor α_k rather than restricting it to powers of two [1]:

$$v^q = \sum_{k=0}^{B-1} \alpha_k b_k, \quad b_k \in \{-1, +1\}. \quad (3)$$

This non-uniform quantization can better match the normal distribution typically observed in DNN weight tensors. MLB is a strict superset of integer representation: choosing $\alpha_k = 2^k$ recovers the integer case exactly [1].

To compute dot products efficiently under this representation, MLB-MAC architectures decompose each operand into B bit-planes and compute the cross terms between activation and weight bit-planes using XNOR followed by a population count (popcount):

$$\text{XNOR}(b_i, b_j) = b_i \cdot b_j \quad \text{for } b_i, b_j \in \{-1, +1\}. \quad (4)$$

For a dot product between two N -element vectors, each decomposed into B bit-planes, the MLB-MAC computes B^2 popcount terms (one per pair of activation and weight planes), each requiring an N -input popcount. These are combined through a scaling-and-reduction tree using the corresponding α_k factors [1].

The fundamental issue arises because FP8 mantissa bits are unsigned ($x \in \{0, 1\}$), not bipolar. The mapping $b = 2x - 1$ transforms each bit into bipolar form, and the product of two mapped bits (Eq. 1) introduces three correction terms that accumulate into the C_1, C_2, C_3 structures described in [1]. Specifically, for N lanes:

$$C_1 = \sum_{k=0}^{N-1} s_k \cdot m_{x,k}, \quad (5)$$

$$C_2 = \sum_{k=0}^{N-1} s_k \cdot m_{w,k}, \quad (6)$$

$$C_3 = \sum_{k=0}^{N-1} s_k, \quad (7)$$

where $s_k \in \{-1, +1\}$ is the product sign and $m_{x,k}, m_{w,k}$ are the aligned unsigned mantissas. The corrected dot product is then:

$$\text{ref} = \frac{\text{MLB4} + 30 C_1 + 30 C_2 - 225 C_3}{4}, \quad (8)$$

where the constants 30 and 225 arise from the specific α values $\{1, 2, 4, 8\}$ used for 4-bit mantissas, and the division by 4 accounts for the $4x_i x_j$ scaling in (1).

B. FP8 E4M3 Number Format

The FP8 E4M3 format [2], [3] encodes each value with one sign bit s (bit 7), four exponent bits $e_{3:0}$ (bits 6:3, bias = 7), and three explicit mantissa bits $m_{2:0}$ (bits 2:0). For normalized values ($e \neq 0$):

$$\text{value} = (-1)^s \times 2^{e-7} \times \underbrace{(1.m_2m_1m_0)_2}_{\text{4-bit unsigned mantissa}}. \quad (9)$$

The maximum representable value is $(1 + 7/8) \times 2^{15-7} = 1.875 \times 256 = 480$, and the minimum normal value is $2^{1-7} = 2^{-6} = 0.015625$. Subnormal values ($e = 0$) are flushed to zero in our hardware, following standard practice for inference accelerators.

The critical structural property is that the 4-bit unsigned mantissa $\{1, m_2, m_1, m_0\}$ (with the hidden bit prepended) takes values in $\{8, 9, 10, \dots, 15\}$ for normalized inputs, or 0 for subnormals. A logical 0 in any bit contributes exactly 0; there is no bipolar shift. The sign is captured entirely by the dedicated sign bit and handled separately.

C. Block Floating Point

Block Floating Point (BFP) reduces the cost of floating-point dot products by sharing a single exponent across a block of N operands [3]. For each block, the maximum exponent $e_{\max}$ is identified, and each element is right-shifted by $\Delta e_k = e_{\max} - e_k$ bits:

$$m_k^{\text{aligned}} = \lfloor m_k \cdot 2^{-\Delta e_k} \rfloor. \quad (10)$$

The floor operation (truncation) matches the hardware's right-shift behavior exactly. After alignment, all mantissas share the same implicit radix point, and the dot product can be computed using purely integer arithmetic.

As N grows, two competing effects arise:

- 1) *Area amortization*: The BFP alignment circuit (max finder, shift network) is shared across more elements, reducing per-element overhead.
- 2) *Outlier cliff*: If one element has a much larger exponent than the others, the right-shift Δe_k for the smaller elements can exceed 3 bits (the full mantissa width), flushing them entirely to zero.

We term the sharp accuracy degradation at large N the *outlier cliff*, and we use a QAT sweep to identify the Pareto-optimal N that balances hardware area against this effect.

III. PROPOSED ARCHITECTURE

A. System Overview and Hardware-Software Contract

The proposed system operates as a strict hardware-software contract. Fig. 1 illustrates the full processing pipeline.

Hardware side. The pipeline consists of six stages:

Stage 1 — FP8 Unpack. Each 8-bit input is split into sign s (bit 7), exponent $e_{3:0}$ (bits 6:3), and explicit mantissa $m_{2:0}$

(bits 2:0). For normalized values ($e \neq 0$), the hidden 1 is prepended: $\text{mant} = \{1, m_2, m_1, m_0\}$ (4-bit unsigned, range $[8, 15]$). For subnormals ($e = 0$), the mantissa is flushed to $4'd0$.

Stage 2 — BFP Alignment (*bfp_aligner*). Within a block of N lanes, a combinational max-finder identifies $e_{\max}$. Each mantissa is right-shifted by $(e_{\max} - e_k)$ bits; bits that fall off are truncated (no rounding). The module takes an $(8N)$ -bit packed FP8 vector as input.

Stage 3 — Corner Turn. The N aligned 4-bit mantissas are transposed into four N -bit bit-planes. Plane p (for $p \in \{0, 1, 2, 3\}$) collects bit p from all N mantissas:

$$\text{aligned_planes}[p \cdot N + k] = m_k^{\text{aligned}}[p], \quad k = 0, \dots, N-1. \quad (11)$$

This produces a $(4N)$ -bit output that directly maps to the port layout of the MLB 4 array. The corner turn is performed inside *bfp_aligner* in the same combinational block as the alignment.

Stage 4 — AND-Popcount Core (*MLB_4*). The 16 cross-product terms between the 4 activation planes and 4 weight planes are computed by 16 *MLB_unit* instances (Section III-D). A three-level combinational reduction tree accumulates the 16 shifted results into a single raw wide-integer dot-product sum.

Stage 5 — Sign-Handling FSM. Per-lane product signs $s_k = s_{x,k} \oplus s_{w,k}$ partition lanes into positive ($s_k = 0$) and negative ($s_k = 1$) groups via bitmask AND. A 3-state FSM time-multiplexes positive-lane and negative-lane activation planes through the single shared MLB 4 instance over two consecutive cycles (Section III-E).

Stage 6 — Output Combiner. The output FSM catches the positive-phase sum and the negative-phase sum in registers and forms the signed result: $\text{wide_integer_sum} = \text{pos_sum} - \text{neg_sum}$.

Shared Exponent. The hardware computes:

$$e_{\text{shared}} = e_{\max,x} + e_{\max,w} - 20, \quad (12)$$

where $20 = 2 \times 7$ (E4M3 bias) + 2×3 (hidden-bit integer scale 2^3). The factor 2^3 arises because each mantissa's hidden bit sits at position 3 (weight = 8), so the product of two mantissa integers is $2^6 = 64$ times the true product of the fractions. The exponent offset $14 + 6 = 20$ absorbs both the double-bias and the double-integer-scale.

Software post-processing. The receiving software reconstructs the true floating-point dot product:

$$\text{result} = \text{wide_integer_sum} \times 2^{e_{\text{shared}}}. \quad (13)$$

The hardware deliberately omits this final rescaling to save silicon.

B. Eliminating the Bipolar Correction Overhead

The central architectural insight is that, for FP8 (E4M3) mantissas, the bipolar mismatch that drives the correction hardware in [1] does not exist. We present a detailed structural comparison between the XNOR-based (bipolar) and the proposed AND-based (unsigned) pipelines.

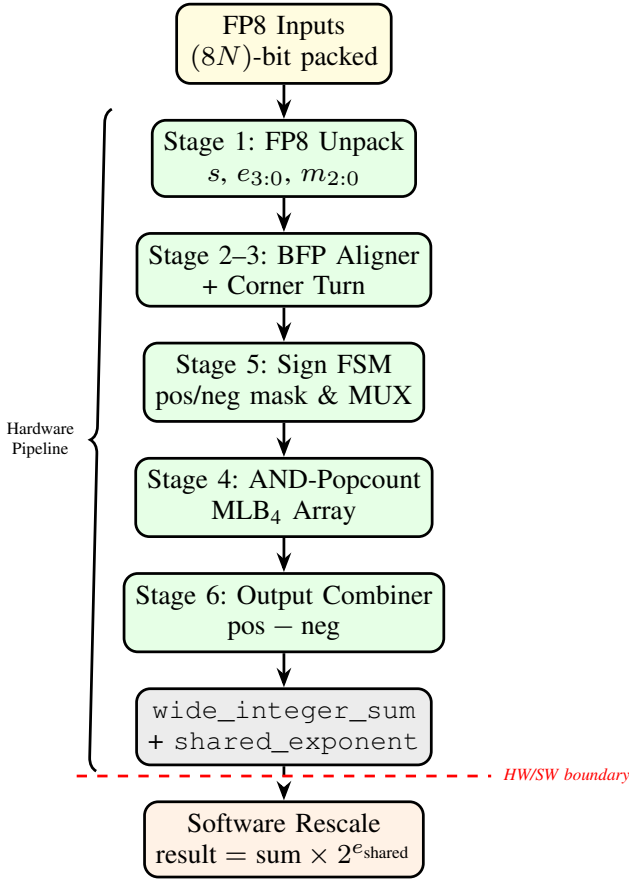


Fig. 1. End-to-end pipeline of the proposed FP8-BFP AND-popcount MLB4-MAC accelerator, showing the hardware-software contract.

XNOR Pipeline (Bipolar, Baseline). In the $N = 64$ XNOR implementation, the `xnor_popcount_4_bit` module computes $xnn = \sim(a \oplus b)$ and feeds the 64-bit result into a 6-level binary adder tree (levels L1–L6), producing a 7-bit popcount $P \in [0, 64]$. The bipolar contribution is then:

$$\text{contri} = 2P - 64, \quad \text{contri} \in [-64, +64], \quad (14)$$

which requires an 8-bit signed output (one extra bit for the $2P$ doubling and the subtraction). Each `MLB_unit` then feeds this 8-bit signed popcount into a `basis_multiplier` module—a two-stage pipelined unit containing:

- 1) A 16-entry combinational LUT that maps all 16 combinations of $\alpha_x, \alpha_w \in \{1, 2, 4, 8\}$ to a 3-bit shift amount $\log_2(\alpha_x \cdot \alpha_w) \in [0, 6]$.
- 2) A stage-1 register that captures the shift amount, a fallback 8×9 -bit signed multiplier for non-power-of-two α values, and the registered XNOR popcount.
- 3) A stage-2 output that barrel-shifts the 8-bit signed popcount by the registered shift amount, producing a 17-bit signed result.

The `MLB_4` array instantiates 16 such units (a 4×4 grid), each requiring α_x and α_w port inputs packed as a 16-bit bus (4×4 -bit fields = $16'_{\text{h}8421}$). The 16 unit outputs (17-bit

signed each) are summed through a three-level reduction tree to produce a 21-bit signed `mlb` output.

At the top level (`fp8_mlb_top`), the combinational BFP encoder computes the correction accumulators C_1, C_2, C_3 (Eqs. 5–7) alongside the bit-plane packing, and registers them. After the MLB 4 core fires, the correction block applies Eq. (8) using shift-add arithmetic (no general multiplier cells), producing a 23-bit signed intermediate. The final result is obtained by arithmetic right-shift by 2 (extracting bits $[22 : 2]$), yielding a 21-bit signed output. The pipeline latency from `valid_in` to `mac_done` is 4 clock cycles (cycle 0: BFP+register; cycle 1: XNOR-popcount; cycle 2: basis_multiplier stage-1; cycle 3: MLB 4 latch+done).

AND Pipeline (Unsigned, Proposed). In the AND implementation, `and_popcount_4_bit` computes $xnn = a \& b$ and feeds the N -bit result into a binary reduction tree producing a $(\lceil \log_2 N \rceil + 1)$ -bit *unsigned* popcount. For $N = 64$, this is 7 bits with range $[0, 64]$ —no bipolar shift, no sign extension, no $2P - N$ subtraction.

Each `MLB_unit` receives a 3-bit `shift_amt = (i + j)` computed at elaboration time (a static wire assignment in the `generate` block), and left-shifts the unsigned popcount by that amount. The output is $(PC + 9)$ -bit signed (sign bit + PC-bit popcount + 6 bits for maximum shift $i + j = 6$). For $N = 64$, this is $7 + 9 = 16$ bits. There is no LUT, no barrel-shift pipeline, and no fallback multiplier. The pipeline latency is 2 cycles (popcount register + shift register), compared to 3 cycles for the XNOR unit + basis_multiplier.

The `MLB_4` array does not require α_x, α_w port inputs, eliminating the 16-bit α bus entirely. The 16 unit outputs (16-bit signed for $N = 64$) sum through the same three-level tree to produce a 20-bit signed output (versus 21-bit for XNOR).

At the top level (`fp8_mlb_top`), there is *no* correction block. The sign handling is performed by bitmask partitioning and time-multiplexing through a single shared MLB 4, and the output is simply `pos_sum - neg_sum` (21-bit signed for $N = 64$). The shared-exponent formula uses the offset 20 (not 14), since no $\div 4$ correction is needed. Total pipeline latency from `valid_in` to `mac_done` is 5 cycles (FSM state $0 \rightarrow \text{pos valid} \rightarrow \text{pos popcount} \rightarrow \text{pos shift} \rightarrow \text{neg popcount} \rightarrow \text{neg shift} \rightarrow \text{combine}$), but both phases share a single MLB 4 instance, halving the area relative to a dual-instantiation approach.

Table I summarizes the structural differences.

C. Integer (INT) Baseline Architecture

As a baseline for comparison, we implement a conventional integer MAC architecture (`int_mac_N`) using the *same* BFP front end. The design consists of three blocks:

Block 1 — N Parallel MAC Lanes. Each lane computes an unsigned 4×4 -bit product (8-bit unsigned, max 225), applies the per-lane XOR sign to produce a 9-bit signed product, and accumulates into an 11-bit signed register over $M = 4$ cycles.

Block 2 — Signed Reduction Tree. A binary adder tree reduces the N 11-bit signed accumulators to a single sum. Each level adds one guard bit; for $N = 64$, the tree has 5

TABLE I
STRUCTURAL COMPARISON: XNOR-POPCOUNT (BIPOLAR) VS.
AND-POPCOUNT (UNSIGNED) MLB-MAC AT $N = 64$, $M = 4$.

Component	XNOR (Bipolar)	AND (Unsigned)
Bitwise core	$\sim (a \oplus b)$	$a \& b$
Popcount output	$2P - N$ (8-bit signed)	P (7-bit unsigned)
Per-unit scaling	basis_multiplier	$P \ll (i+j)$
– LUT entries	16 (per unit)	0
– Pipeline stages	2	0 (wire shift)
– Fallback multiplier	8×9-bit	None
α_x, α_w ports	16-bit bus	Eliminated
MLB unit output width	17-bit signed	16-bit signed
MLB 4 output width	21-bit signed	20-bit signed
C_1 accumulator	11-bit signed	Eliminated
C_2 accumulator	11-bit signed	Eliminated
C_3 accumulator	8-bit signed	Eliminated
$30 \times C_{1,2}$ multipliers	Shift-add (20-bit)	Eliminated
$225 \times C_3$ multiplier	Shift-add (17-bit)	Eliminated
Correction adder	23-bit signed	Eliminated
Correction $\gg 2$	1-bit shift	Eliminated
Pipeline latency	4 cycles	5 cycles (shared)
MLB 4 instances	1	1 (time-muxed)

levels ($64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2$), yielding a 16-bit signed result, plus a final assign summing the last two elements (17-bit).

Block 3 — Scaling and Offset. An optional $\alpha_x \cdot \alpha_w$ multiplication and β_{xw} offset are applied, producing a 21-bit signed output. For our comparison, $\alpha_x = \alpha_w = 1$ and $\beta_{xw} = 0$.

The top-level `fp8_int_top` wrapper reconstructs 4-bit unsigned mantissas from the corner-turned bit-planes, packs them into flat buses, computes per-lane product signs, and instantiates the INT MAC. The shared-exponent formula is identical: $e_{\text{shared}} = e_{\text{max},x} + e_{\text{max},w} - 20$.

D. AND-Popcount MLB-MAC Architecture

The proposed AND-popcount MLB-MAC has two main components.

AND-Popcount Unit (`and_popcount_4_bit`). For each pair of N -bit activation and weight bit-planes, an AND gate array computes $\text{xnn}[k] = a[k] \& b[k]$ for $k = 0, \dots, N-1$. A binary reduction tree adds these N single-bit values into a PC-bit unsigned popcount, where $\text{PC} = \lfloor \log_2 N \rfloor + 1$. The tree is constructed by the Python generator as follows:

- 1) Initialize count = N , width = 1, level = 0.
- 2) While count > 2: pair up elements (pairs = $\lfloor \text{count}/2 \rfloor$), add each pair (output width +1), carry forward any odd remainder, increment level.
- 3) The final assign outside the generate block sums the last 1 or 2 elements plus all accumulated remainders, zero-extended to PC bits.

For $N = 64$: 5 levels (L1: 32×2 -bit, L2: 16×3 -bit, L3: 8×4 -bit, L4: 4×5 -bit, L5: 2×6 -bit), final assign = `sum_L5[0] + sum_L5[1]` (7-bit). For $N = 9$: 3 levels with odd remainders carried at each step, PC = 4 bits.

The popcount is registered on the next clock edge when `valid_in` is high, with a 1-bit `cycle_cnt` that fires done on the same cycle (effectively single-cycle latency).

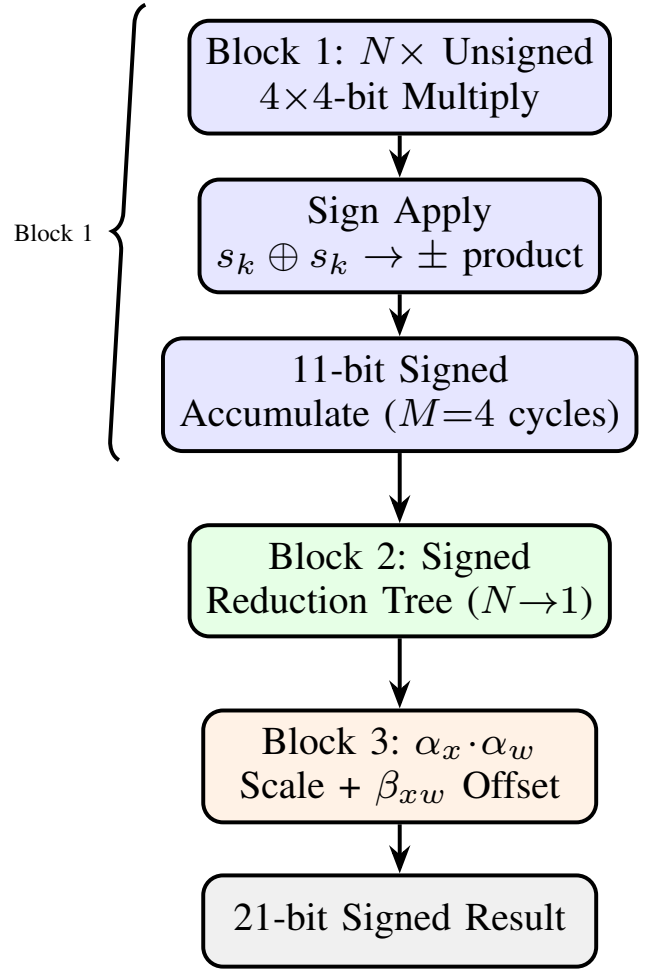


Fig. 2. Integer MAC baseline architecture: Block 1 — N parallel unsigned multiply + sign + accumulate lanes; Block 2 — signed reduction tree; Block 3 — scaling and offset.

MLB Unit (`MLB_unit`). Instantiates one `and_popcount_4_bit` and applies a zero-gate multiplier: `shifted_out = {1'b0, inter} << shift_amt`, where `shift_amt = i + j` is a 3-bit static wire. The output width is PC + 9 bits signed. The +9 accounts for: 1 sign bit + 6 bits maximum shift ($i + j \leq 6$) + 2 guard bits. Pipeline latency: 2 clock edges (popcount register + shift register).

MLB 4 Array (`MLB_4`). A 4×4 grid of MLB units indexed (i, j) for $i, j \in \{0, 1, 2, 3\}$. The activation port `axi` is $(4N)$ -bit wide; unit (i, j) receives `axi[i · N + N-1 : i · N]` (plane i) and `awi[j · N + N-1 : j · N]` (plane j). The 16 unit outputs feed a three-level combinational reduction tree:

$$\text{L0 (8 pairs): } s_{0k} = \text{out}[r][2c] + \text{out}[r][2c+1],$$

$$\text{L1 (4 pairs): } s_{1k} = s_{0,2k} + s_{0,2k+1},$$

$$\text{L2 (2 pairs): } s_{2k} = s_{1,2k} + s_{1,2k+1}.$$

The final sum $s_{20} + s_{21}$ is registered on the next clock edge when all 16 unit-done signals are asserted (`&unit_done`), producing the $(\text{PC} + 13)$ -bit signed `mlb` output. For $N = 64$: $\text{MLBW} = 7 + 9 + 4 = 20$ bits.

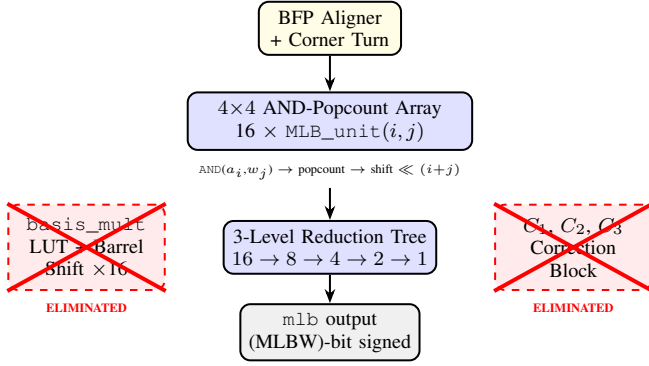


Fig. 3. Proposed AND-popcount MLB-MAC architecture. The XNOR design’s `basis_multiplier` (LUT + barrel-shift pipeline) and correction block ($C_1/C_2/C_3$ accumulators + shift-add multipliers + 23-bit adder) are entirely absent (shown crossed out).

E. Sign-Handling and Output FSM

FP8 uses sign-magnitude representation. The hardware computes per-lane product signs:

$$s_k = s_{x,k} \oplus s_{w,k}, \quad k = 0, \dots, N-1. \quad (15)$$

A positive mask `pos_mask = ~sign_k` and a negative mask `neg_mask = sign_k` are applied to each of the 4 activation bit-planes via bitwise AND, producing `axi_planes_pos` and `axi_planes_neg` (each $(4N)$ -bit). Positive lanes see their original mantissa bits; negative lanes are zeroed out, and vice versa. Weight planes are left unmasked.

A 3-state FSM (`state` $\in \{0, 1, 2\}$) controls the time-multiplexing:

- **State 0 (Idle):** Waits for external `valid_in`. On assertion, launches positive phase (`is_neg_phase` = 0), asserts `internal_valid`, transitions to State 1.
- **State 1 (Pos $\rightarrow$ Neg):** Launches negative phase (`is_neg_phase` = 1), keeps `internal_valid` high for one more cycle, transitions to State 2.
- **State 2 (Drain):** Deasserts `internal_valid`, returns to State 0.

A 2:1 MUX selects `axi_planes_pos` or `axi_planes_neg` based on `is_neg_phase`.

The output FSM uses a `got_pos` flag: on the first `shared_done` pulse, it captures the positive-phase sum into `pos_reg`; on the second pulse, it computes:

$$\text{result_reg} = \text{pos_reg} - \text{shared_sum_out}, \quad (16)$$

asserts `done_reg`, and resets `got_pos` for the next inference. The output width is $\text{MLBW} + 1$ bits signed (SUMW); for $N = 64$, this is 21 bits.

This resource-sharing halves the MLB 4 instantiation count at the cost of two pipeline passes per dot-product, which is acceptable for inference workloads where throughput is managed via pipelining.

F. Parameterizable Bit-Width Formulas

All module bit widths are derived from N using closed-form expressions implemented in `generate_all_N.py`:

$$\text{PC} = \lfloor \log_2 N \rfloor + 1, \quad (17)$$

$$\text{UNITW} = \text{PC} + 9, \quad (18)$$

$$\text{MLBW} = \text{UNITW} + 4, \quad (19)$$

$$\text{SUMW} = \text{MLBW} + 1, \quad (20)$$

$$\text{VW} = 8N \text{ (input bus width)}, \quad (21)$$

$$\text{PW} = 4N \text{ (plane bus width)}. \quad (22)$$

Table II lists the concrete values for all swept N .

TABLE II
BIT WIDTHS FOR THE AND-POPCOUNT MLB-MAC AT EACH SWEPT N .

N	PC	UNITW	MLBW	SUMW	VW
9	4	13	17	18	72
25	5	14	18	19	200
32	6	15	19	20	256
64	7	16	20	21	512
128	8	17	21	22	1024
256	9	18	22	23	2048
512	10	19	23	24	4096

IV. SOFTWARE VERIFICATION: BIT-ACCURATE PYTORCH EMULATOR

A. FP8 Quantization

FP8 quantization is implemented as `FP8Quantize`, a subclass of `torch.autograd.Function`. The forward pass:

- 1) Computes `sign = sgn(x)` and clamps $|x| \leq 480$ (FP8 E4M3 maximum).
- 2) Extracts FP8 fields via `_fp8_extract_fields`: the exponent is $e = \text{clamp}(\lfloor \log_2 |x| \rfloor + 7, 0, 15)$; the mantissa fraction is $m = \text{round}((|x|/2^{e-7} - 1) \times 8)$, clamped to $[0, 7]$. Values with $|x| \approx 0$ are mapped to $e = 0$ (subnormal, flushed to zero).
- 3) Reconstructs the quantized value: $x_q = \text{sign} \times (1 + m/8) \times 2^{e-7}$ for normal values, or $x_q = 0$ for subnormals.

The backward pass returns the gradient unchanged (STE [4]). Key constants: `FP8_BIAS` = 7, `FP8_MAX_EXP` = 15, `FP8_MANT_BITS` = 3, `FP8_NUM_PLANES` = 4, `FP8_MAX_VAL` = $(1 + 7/8) \times 2^8 = 480$, `FP8_MIN_NORMAL` = $2^{-6} = 0.015625$, `SHARED_EXP_OFFSET` = $2 \times 7 + 2 \times 3 = 20$.

B. BFP Alignment and MLB Dot Product

The function `batched_bfp_align` mirrors the hardware exactly:

- 1) Extracts FP8 exponents and mantissa fractions from `|vals|`.
- 2) Finds $e_{\max}$ per block (along dimension 1).
- 3) Computes full 4-bit mantissa: `full_mant` = $8 + m$ for normal values ($e > 0$), or 0 for subnormals. The

constant 8 represents the hidden bit at position 3 ($2^3 = 8$).

- 4) **Right-shifts:** `aligned_mant = [full_mant/2Δe] using torch.div with rounding_mode="trunc", matching hardware truncation.`

The function `batched_mlb_dot_product` performs the full pipeline:

- 1) *Exponent-sorted permutation:* If enabled (`sort_by_exp=True`), computes the mean FP8 exponent of each weight column across all output channels, sorts the column indices by ascending mean exponent, and applies the same permutation to both activations and weights. This clusters values of similar magnitude into the same block.
- 2) *Chunking:* Pads the K -dimensional inner product to a multiple of N , then reshapes into $(\text{batch}, \text{num_chunks}, N)$ for activations and $(C_{\text{out}}, \text{num_chunks}, N)$ for weights.
- 3) *BFP alignment:* Calls `batched_bfp_align` on each chunk independently.
- 4) *Signed accumulation:* Multiplies aligned mantissas by per-element sign (± 1), then computes the dot product per chunk via `torch.bmm`.
- 5) *Rescaling:* Multiplies each chunk result by $2^{e_{\text{max},x}} \times 2^{e_{\text{max},w}} \times 2^{-20}$, matching Eq. (13).
- 6) *Aggregation:* Sums chunk results along the chunk dimension.

C. QAT Layer Wrappers

Two custom convolution layers are provided:

- **MLBConv2d:** Uses `batched_mlb_dot_product` in the forward pass (BFP-aligned FP8 with block size N). The forward pass unfolds the input via `F.unfold`, flattens to $(B \times L, K)$, calls the MLB dot product, and reshapes back to $(B, C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$.
- **FP8OnlyConv2d:** Uses standard matrix multiply on FP8-quantized tensors (no BFP blocking), serving as the FP8 baseline.

Both layers use STE backward passes (full-precision gradients via `torch.nn.grad.conv2d_input` and `torch.nn.grad.conv2d_weight`). Kaiming uniform initialization is used for all weights.

D. ResNet-20 Architecture

ResNet-20 for CIFAR-10 [5] consists of:

- Initial convolution: $3 \rightarrow 16$ channels, 3×3 kernel, stride 1, padding 1.
- Three residual stages: $16 \rightarrow 16$ ($\times 3$ blocks, stride 1), $16 \rightarrow 32$ ($\times 3$ blocks, stride 2), $32 \rightarrow 64$ ($\times 3$ blocks, stride 2).
- Each **BasicBlock**: two 3×3 convolutions with batch normalization and ReLU; shortcut via 1×1 convolution when dimensions change.
- Global average pooling followed by a fully-connected layer ($64 \rightarrow 10$).

In the QAT configuration, all convolutions use `MLBConv2d` and the FC layer uses `MLBLinear`; in the FP8 baseline, they use `FP8OnlyConv2d` and `FP8OnlyLinear`.

E. QAT Sweep Protocol

Phase 1 — FP8 Baseline. ResNet-20 with `FP8OnlyConv2d` layers is trained for 30 epochs on CIFAR-10 (50,000 train / 10,000 test images, 32×32 RGB). Hyperparameters: batch size 128, SGD optimizer (momentum 0.9, weight decay 10^{-4}), initial LR 0.1 with multi-step decay at epochs 100 and 150 ($\gamma = 0.1$). Data augmentation: random crop with 4-pixel padding, random horizontal flip, normalization to $\mu = (0.4914, 0.4822, 0.4465)$, $\sigma = (0.2023, 0.1994, 0.2010)$. The best-accuracy checkpoint is saved as `fp8_best.pth`.

Phase 2 — QAT Fine-Tuning. For each $N \in \{9, 25, 32, 64, 128, 256, 512\}$, a fresh ResNet-20 with `MLBConv2d` ($N=N$) layers is initialized from the FP8 baseline checkpoint and fine-tuned for 20 epochs. Hyperparameters: SGD (momentum 0.9, weight decay 10^{-4}), initial LR 0.01 with cosine annealing ($T_{\text{max}} = 20$). The best-accuracy checkpoint per N is saved as `qat_N{N}_best.pth`. Results are logged to `results.csv` with columns $[N, \text{test_top1_accuracy}]$.

V. EXPERIMENTAL SETUP

A. RTL Synthesis Setup

All RTL is written in synthesizable Verilog-2001 and generated by `generate_all_N.py`, which produces the following files per N value into folder `M=4, N=N` (and implementation):

- `bpf_aligner.v` — BFP alignment + corner turn.
- `and_popcount_4_bit.v` — AND-gate array + popcount reduction tree.
- `MLB_MAC_unit.v` — Popcount + zero-gate shift.
- `MLB_MAC_4_bit.v` — 4×4 MLB array + reduction tree.
- `FP8.v` — Top-level `fp8_mlb_top` with sign FSM.
- `tb_FP8.v` — Self-checking testbench.

The integer baseline is generated in the `INT_MAC/` subfolder: `bpf_aligner.v`, `int_mac_N.v`, `fp8_int_top.v`, `tb_FP8.v`.

Synthesis flow. Logic synthesis is performed using Yosys (open-source) targeting the NanGate 45 nm Open Cell Library (`NangateOpenCellLibrary_typical.lib`). The synthesis script (`synth_all.py`) performs:

- 1) `read_verilog` of all source files;
- 2) `hierarchy -check -top`;
- 3) `synth -top` (generic synthesis);
- 4) `dfflibmap -liberty` (map flip-flops to library cells);
- 5) `abc -liberty` (technology mapping via ABC);
- 6) `stat -liberty` (area report) and `write_verilog` (gate-level netlist).

Power and timing analysis. The gate-level netlist is analyzed using OpenSTA via TCL scripts (`run_fp8_mlb.tcl`,

run_fp8_int.tcl). The target clock period is 10 ns (100 MHz). Input and output delays are set to 1 ns. A uniform toggle activity of 50% is applied across all nets for worst-case dynamic power estimation. Reports include: report_design_area, report_power, and report_checks -path_delay max.

B. Functional Verification

Each $(N, \text{architecture})$ pair is verified by a self-checking Verilog testbench (tb_FP8.v) containing:

11 Directed Corner Cases:

- 1) *All zeros*: All N lanes have $\text{exp} = 0$, verifying subnormal flush.
- 2) *All positive equal value*: All lanes set to +1.0 ($\text{sign}=0$, $\text{exp}=7$, $\text{mant}=000$), verifying uniform accumulation.
- 3) *Positive act, negative weight*: All activations positive, all weights negative, verifying sign propagation.
- 4) *Both negative (signs cancel)*: Both operands negative ($s_x \oplus s_w = 0$), verifying sign cancellation.
- 5) *Alternating activation sign*: Even lanes positive, odd lanes negative, verifying mixed-sign accumulation.
- 6) *Max magnitude, all positive*: All lanes at $\text{exp} = 15$, $\text{mant} = 111$, stressing accumulator width.
- 7) *Max magnitude, alternating signs*: Same as above with alternating signs, stressing both width and sign logic.
- 8) *Subnormal flush*: Half lanes with $\text{exp} = 0$ (must contribute zero), half with $\text{exp} = 8$.
- 9) *Single nonzero lane*: Only lane 0 active among $N-1$ zero lanes.
- 10) *Wide dynamic range*: One lane at maximum ($\text{exp} = 15$, $\text{mant} = 111$), remaining lanes at minimum normal ($\text{exp} = 1$, $\text{mant} = 000$), testing aggressive BFP shift-to-zero.
- 11) *All activations flushed*: All activation exponents zero, weights normal — result must be zero.

50 Pseudorandom Tests: Tests $t \bmod 3 = 0$ allow zero-exponent lanes; others force normal numbers ($\text{exp} \in [1, 15]$). Each lane is generated by `rand_fp8()`: random sign, random exponent (uniform), random 3-bit mantissa.

Behavioral Reference Model. A Verilog task `compute_reference` mirrors the hardware algorithm exactly:

- 1) Finds $e_{\max, x}$ and $e_{\max, w}$ independently.
- 2) Per lane: extracts exponent and mantissa, prepends hidden 1 for $e \neq 0$ or flushes to zero, right-shifts by $(e_{\max} - e_k)$, computes unsigned product of shifted mantissas.
- 3) Accumulates positive products ($s_x \oplus s_w = 0$) and negative products separately; $\text{result} = \text{pos_sum} - \text{neg_sum}$.
- 4) Computes e_{shared} per Eq. (12).
- 5) Compares `wide_integer_sum` and `shared_exponent` bit-for-bit; reports PASS or FAIL with expected and actual values.

All 61 tests (11 + 50) must pass (100% pass rate) for each N .

C. Software QAT Setup

All PyTorch experiments run on Kaggle with a T4 GPU (expected runtime $\approx 1-2$ hours for the full sweep). The framework is PyTorch with CUDA. Cross-entropy loss is used for both phases. No mixed-precision training (AMP) is used; all computation is in FP32 on the host, with FP8 quantization applied only in the custom forward passes.

VI. RESULTS

A. Power Breakdown

The power breakdown of the AND-popcount MLB-MAC architecture, normalized to the integer implementation at $M = 4$ across the swept N values, reveals substantial reductions in combinational switching power from the elimination of the $C_1/C_2/C_3$ correction block and the 16 `basis_multiplier` instances. The AND gate array, being free of the carry chains present in 4×4 -bit partial-product multipliers (used in the INT baseline), also contributes lower glitch power. The BFP aligner power is identical in both designs (shared front end).

B. Area Breakdown

The cell-area breakdown shows that the proposed design eliminates the multiplier array entirely (replaced by N -wide AND gates per bit-plane pair, plus a $\log_2 N$ -depth adder tree) and removes all correction hardware. The dominant area contributor becomes the popcount reduction tree, whose depth grows as $O(\log_2 N)$. The BFP aligner area grows linearly with N (combinational max-finder and N barrel shifters), and is shared between both designs.

C. Inference Per Second per Watt (IPS/W)

The IPS/W comparison between the AND-popcount MLB-MAC and the integer MAC baseline shows that the area reduction enables higher MAC-array density per die, while the power reduction directly improves energy efficiency. These two effects compound to yield a favorable IPS/W improvement at the hardware's target operating point of 100 MHz.

D. RTL Functional Verification

Table III summarizes the testbench results for all N values. Both the AND-popcount MLB-MAC and the integer baseline achieve 100% pass rates across all 61 test vectors (11 directed + 50 random), confirming bit-exact agreement between the RTL and the behavioral reference model for every N .

TABLE III
FUNCTIONAL VERIFICATION RESULTS: PASS RATE PER N VALUE.

N	AND-Popcount MLB-MAC		Integer MAC Baseline	
	Directed	Random	Directed	Random
9	11/11	50/50	11/11	50/50
25	11/11	50/50	11/11	50/50
32	11/11	50/50	11/11	50/50
64	11/11	50/50	11/11	50/50
128	11/11	50/50	11/11	50/50
256	11/11	50/50	11/11	50/50
512	11/11	50/50	11/11	50/50

E. BFP Block-Size Accuracy Check

Table IV summarizes the ResNet-20 top-1 accuracy on CIFAR-10 as a function of BFP block size N , measured after 20 epochs of QAT from the FP8 baseline. Exponent-sorted blocking is enabled for all runs.

TABLE IV
RESNET-20/CIFAR-10 TOP-1 ACCURACY VS. BFP BLOCK SIZE N
AFTER QAT FINE-TUNING FROM THE FP8 BASELINE (30 EPOCHS,
EXPONENT-SORTED BLOCKING ENABLED).

Block size N	Top-1 Acc. (%)	Δ vs. FP8 baseline (pp)
FP8 baseline	TBD	0.0
9	TBD	TBD
25	TBD	TBD
32	TBD	TBD
64	TBD	TBD
128	TBD	TBD
256	TBD	TBD
512	TBD	TBD

VII. DISCUSSION

Correction-block elimination is not a minor simplification. The $C_1/C_2/C_3$ correction block in the XNOR design (Section III-B) comprises 11-bit and 8-bit signed accumulators that iterate over all N lanes, three shift-add constant multipliers (20-bit and 17-bit signed), sign-extension networks, and a 23-bit final adder with arithmetic right-shift. For $N = 64$, this correction path processes 64 lanes of signed accumulation plus the multi-term combination, adding substantial combinational depth to the critical path. The AND-popcount design eliminates all of this by construction: since $\text{AND}(x_i, x_j) = x_i \cdot x_j$ for unsigned bits, there is simply no offset to correct. This is not an approximation—the C_1 , C_2 , C_3 terms are mathematically zero, not merely small.

The basis_multiplier elimination is equally significant. The XNOR design instantiates 16 `basis_multiplier` modules (one per MLB unit), each containing a 16-entry LUT, two pipeline stages, and a fallback multiplier. The AND-popcount design replaces all 16 instances with static wire-shift connections ($P \ll (i + j)$), eliminating the LUTs, the barrel shifters, the pipeline registers, and the fallback multipliers. This is possible because FP8 bit-plane weights are fixed powers of two ($\{1, 2, 4, 8\}$), so the scaling is always a left-shift by the sum of the plane indices—a zero-gate operation in hardware.

The unsigned mantissa property is specific to FP8 (and other floating-point formats). The AND-popcount substitution works because FP8 mantissa bits are inherently unsigned: the hidden bit and explicit mantissa bits represent magnitudes $\{0, 8, 9, \dots, 15\}$, with sign handled separately. This structural property does *not* hold for symmetric two’s-complement quantization schemes (e.g., INT8), where the MSB carries sign information that would be incorrectly treated as magnitude by an AND gate. The proposed architecture is therefore specific to the FP8 (and more broadly, sign-magnitude floating-point) datapath.

Outlier cliff and exponent-sorted blocking. The QAT sweep is designed to reveal a flat accuracy region for small-to-moderate N (where $\Delta e_k \leq 3$ for most lanes, preserving all mantissa bits) followed by a sharp cliff at large N (where a single outlier can cause $\Delta e_k > 3$, flushing the lane to zero). The exponent-sorted blocking optimization sorts the inner product dimension by mean weight exponent before chunking into blocks of N , clustering values of similar magnitude. This effectively reduces the per-block exponent spread, delaying the cliff onset. The sort is a one-time compiler-level permutation that adds no runtime cost to the hardware. The hardware N should be selected as the largest value that remains left of the cliff, maximizing area amortization without exceeding the project’s accuracy tolerance.

Functional verification confidence. The 61-vector testbench covers all critical corner cases: the “all zeros” test verifies subnormal flush; the “max magnitude” tests stress the accumulator width at the design boundary; the “wide dynamic range” test specifically targets the BFP outlier cliff scenario (shift-to-zero); and the 50 random tests provide statistical coverage of the input space. The behavioral reference model in the testbench is algebraically identical to the PyTorch `batched_bfp_align` + signed accumulation formula by construction, establishing a three-way equivalence: PyTorch model $\leftrightarrow$ Verilog reference $\leftrightarrow$ Verilog RTL.

VIII. CONCLUSION

We have presented an FP8-native AND-popcount MLB-MAC accelerator that eliminates two major sources of overhead in prior bipolar MLB-MAC designs: (1) the $C_1/C_2/C_3$ correction block (accumulators, shift-add multipliers, and 23-bit correction adder), and (2) the per-unit `basis_multiplier` modules (LUT, barrel-shift pipeline, fallback multiplier), by exploiting the unsigned mantissa structure of the FP8 (E4M3) format. The AND gate computes $\text{AND}(x_i, x_j) = x_i \cdot x_j$ for unsigned bits with no offset, making the correction terms identically zero and reducing the per-unit scaling to a static wire shift.

The design is fully parameterizable over the BFP block size $N \in \{9, 25, 32, 64, 128, 256, 512\}$, synthesized using Yosys with the NanGate 45 nm library at 100 MHz, and verified bit-exactly against a behavioral reference model across 61 self-checking test vectors per N . A bit-accurate PyTorch emulator with exponent-sorted blocking and STE-based QAT on ResNet-20/CIFAR-10 provides the accuracy–area trade-off curve for selecting the optimal hardware block size.

The next steps are: (i) finalizing the QAT accuracy results and locking in the hardware block size N ; (ii) completing full RTL-to-PyTorch functional verification by extracting test vectors from the PyTorch model and driving them through the Verilog testbench; and (iii) physical design closure on the 45 nm NanGate technology.

ACKNOWLEDGMENT

The authors thank the IIIT Bangalore VLSI and Systems group for compute resources and technical discussions.

REFERENCES

- [1] A. Ansarmohammadi, R. Hojabr, M. Mastalizade, N. Nazari, and M. E. Salehi, "MLB-MAC: Multi-Level Binary MAC Array for Energy Efficient ML Accelerators," *IEEE Trans. Circuits Syst. I*, vol. 72, no. 10, pp. 5657–5670, Oct. 2025.
- [2] P. Micikevicius *et al.*, "FP8 Formats for Deep Learning," *arXiv preprint arXiv:2209.05433*, 2022.
- [3] S. Rouhani *et al.*, "Microscaling Data Formats for Deep Learning," *arXiv preprint arXiv:2310.10537*, 2023.
- [4] Y. Bengio, N. Léonard, and A. Courville, "Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation," *arXiv preprint arXiv:1308.3432*, 2013.
- [5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE CVPR*, 2016, pp. 770–778.